Get Next Line

Читать строку из файлового дескриптора — слишком утомительно.

Краткое описание: в этом задании нужно написать функцию, которая возвращает строку, прочитанную из файлового дескриптора.

Версия: 14.3

Содержание

I	Цели
II	Общие инструкции
III	AI Instructions
IV	Mandatory part
V	Требования к README
VI	Бонусная часть
VII	Сдача и взаимная проверка

Глава I — Цели

Это задание не только позволит вам добавить очень полезную функцию в вашу коллекцию, но и научит важному понятию программирования на `C`: статическим переменным.

Глава II — Общие инструкции

- Ваш проект должен быть написан на C.
- Ваш проект должен быть написан в соответствии с Нормой (Norm). Если у вас есть бонусные файлы/функции, они тоже входят в проверку нормы, и вы получите `0`, если будет найдена ошибка нормы.
- Ваши функции не должны неожиданно завершаться (segmentation fault, bus error, double free и т. д.), за исключением случаев неопределённого поведения. Если это произойдёт, ваш проект будет считаться нерабочим и получит `0` на проверке.
- Вся память, выделенная в куче, должна быть корректно освобождена, когда это необходимо. Утечки памяти недопустимы.
- Если задание этого требует, вы должны сдать `Makefile`, который компилирует ваши исходные файлы в требуемый результат с флагами `-Wall`, `-Wextra` и `-Werror`, используя `cc`. Кроме того, ваш `Makefile` не должен выполнять лишнюю перелинковку (relink).
- Ваш `Makefile` должен содержать как минимум правила `$(NAME)`, `all`, `clean`, `fclean` и `re`.
- Чтобы сдать бонусы, вы должны добавить в `Makefile` правило `bonus`, которое подключит все заголовки, библиотеки или функции, запрещённые в основной части проекта. Бонусы должны располагаться в файлах `_bonus.{c/h}`, если в задании не указано иное. Обязательная и бонусная части оцениваются отдельно.
- Если проект разрешает использовать вашу `libft`, вы должны скопировать её исходники и соответствующий `Makefile` в папку `libft`. `Makefile` вашего проекта должен сначала собрать библиотеку с помощью её `Makefile`, а затем собрать проект.
- Мы рекомендуем вам писать тестовые программы для своего проекта, хотя эту работу **не нужно сдавать и она не оценивается**. Это даст вам возможность легко проверить свою работу и работу коллег. Такие тесты особенно пригодятся во время защиты. Во время защиты вы можете свободно использовать свои тесты и/или тесты того, кого вы проверяете.
- Сдавайте работу в назначенный Git-репозиторий. Оцениваться будет только то, что находится в Git-репозитории. Если вашу работу проверяет Deepthought, это произойдёт после взаимных проверок. Если во время проверки Deepthought в какой-либо части вашей работы возникнет ошибка, проверка будет остановлена.

Chapter III — AI Instructions

Context

This project is designed to help you discover the fundamental building blocks of your 42 training.

To properly anchor key knowledge and skills, it's essential to adopt a thoughtful approach to using AI tools and support.

True foundational learning requires genuine intellectual effort — through challenge, repetition, and peer-learning exchanges.

For a more complete overview of our stance on AI — as a learning tool, as part of the 42 training, and as an expectation in the job market — please refer to the dedicated FAQ on the intranet.

Main message

- Build strong foundations without shortcuts.
- Really develop tech & power skills.
- Experience real peer-learning, start learning how to learn and solve new problems.
- The learning journey is more important than the result.
- Learn about the risks associated with AI, and develop effective control practices and countermeasures to avoid common pitfalls.

Learner rules:

- You should apply reasoning to your assigned tasks, especially before turning to AI.
- You should not ask for direct answers to the AI.
- You should learn about 42 global approach on AI.

Phase outcomes:

Within this foundational phase, you will get the following outcomes:

- Get proper tech and coding foundations.
- Know why and how AI can be dangerous during this phase.

Comments and example:

- Yes, we know AI exists — and yes, it can solve your projects. But you're here to learn, not to prove that AI has learned. Don't waste your time (or ours) just to demonstrate that AI can solve the given problem.
- Learning at 42 isn't about knowing the answer — it's about developing the ability to find one. AI gives you the answer directly, but that prevents you from building your own reasoning. And reasoning takes time, effort, and involves failure. The path to success is not supposed to be easy.
- Keep in mind that during exams, AI is not available — no internet, no smartphones, etc. You'll quickly realise if you've relied too heavily on AI in your learning process.
- Peer learning exposes you to different ideas and approaches, improving your interpersonal skills and your ability to think divergently. That's far more valuable than just chatting with a bot. So don't be shy — talk, ask questions, and learn together!
- Yes, AI will be part of the curriculum — both as a learning tool and as a topic in itself. You'll even have the chance to build your own AI software. In order to learn more about our crescendo approach you'll go through in the documentation available on the intranet.

✓ Good practice:

> I'm stuck on a new concept. I ask someone nearby how they approached it. We talk for 10 minutes — and suddenly it clicks. I get it.

✗ Bad practice:

> I secretly use AI, copy some code that looks right. During peer evaluation, I can't explain anything. I fail. During the exam — no AI — I'm stuck again. I fail.

Chapter IV — Mandatory part

Function Name	`get_next_line`
Prototype	`char *get_next_line(int fd);`
Files to Submit	`get_next_line.c`, `get_next_line_utils.c`, `get_next_line.h`, all at the root of the repository
Parameters	`fd`: The file descriptor to read from
Return Value	Read line: correct behavior `NULL`: there is nothing else to read, or an error occurred
External Function	`read`, `malloc`, `free`
Description	Write a function that returns a line read from a file descriptor

- Repeated calls (e.g., using a loop) to your `get_next_line()` function should let you read the text file pointed to by the file descriptor, **one line at a time**.
- Your function should return the line that was read.
  If there is nothing left to read or if an error occurs, it should return `NULL`.
- Make sure that your function works as expected both when reading a file and when reading from the standard input.
- **Please note** that the returned line should include the terminating `\n` character, except when the end of the file is reached and the file does not end with a `\n` character.
- Your header file `get_next_line.h` must at least contain the prototype of the `get_next_line()` function.
- Add all the helper functions you need in the `get_next_line_utils.c` file.

> ℹ️ A good start would be to know what a `static variable` is.

- Because you will have to read files in `get_next_line()`, add this option to your compiler call: `-D BUFFER_SIZE=n`
  It will define the buffer size for `read()`.
  The buffer size value will be adjusted by your peer evaluators and the Moulinette to test your code.

> ⚠️ We must be able to compile this project with and without the `-D BUFFER_SIZE` flag in addition to the usual flags. You may choose any default value you prefer.

- You will compile your code as follows (a buffer size of 42 is used as an example):
  `cc -Wall -Wextra -Werror -D BUFFER_SIZE=42 <files>.c`
  (Note: the files must be at the root of the repository).
- `get_next_line()` exhibits undefined behavior if the file associated with the file descriptor is modified after the last call, while `read()` has not yet reached the end of the file.
- `get_next_line()` also exhibits undefined behavior when reading a binary file. However, you can implement a logical way to handle this behavior if you want to.

> 💡 Does your function still work if the BUFFER_SIZE value is 9999? If it is 1? 10000000? Do you know why?

> ℹ️ Try to read as little as possible each time `get_next_line()` is called. If you encounter a new line, you have to return the current line.
> Don't read the whole file and then process each line.

Forbidden

- You are not allowed to use your `libft` in this project.
- `lseek()` is forbidden.
- Global variables are forbidden.

Глава V — Требования к README

В корне вашего Git-репозитория должен находиться файл `README.md`. Его цель — позволить любому, кто не знаком с проектом (коллегам, сотрудникам, рекрутерам и т. д.), быстро понять, о чём проект, как его запустить и где найти дополнительную информацию по теме.

`README.md` должен содержать как минимум:

- Самая первая строка должна быть набрана курсивом и гласить: *This project has been created as part of the 42 curriculum by <login1>[, <login2>[, <login3>[...]]].*
- Раздел «**Description**», в котором чётко представлен проект, включая его цель и краткий обзор.
- Раздел «**Instructions**», содержащий всю необходимую информацию о компиляции, установке и/или запуске.
- Раздел «**Resources**» со списком классических источников по теме (документация, статьи, руководства и т. д.), а также описанием того, как использовался ИИ — с указанием, для каких задач и каких частей проекта.

➠ **В зависимости от проекта могут потребоваться дополнительные разделы** (например, примеры использования, список возможностей, технические решения и т. д.).

Все обязательные дополнения будут явно перечислены ниже.

- Также должно быть приведено подробное объяснение и обоснование алгоритма, выбранного для этого проекта.

> ℹ️ Рекомендуется английский язык; в качестве альтернативы можно использовать основной язык вашего кампуса.

Глава VI — Бонусная часть

Этот проект простой и не предполагает сложных бонусов. Однако мы верим в вашу изобретательность. Если вы выполнили обязательную часть, попробуйте выполнить и бонусную.

Требования к бонусной части:

- Реализуйте `get_next_line()`, используя только одну статическую переменную.
- Ваша `get_next_line()` должна уметь работать с несколькими файловыми дескрипторами одновременно.
  Например, если вы читаете из файловых дескрипторов 3, 4 и 5, вы должны иметь возможность при каждом вызове читать из разных дескрипторов, не теряя состояние чтения каждого из них и не возвращая строку из другого дескриптора.
  Это означает, что вы должны иметь возможность вызвать `get_next_line()` для чтения из fd 3, затем из fd 4, затем из fd 5, затем снова из fd 3, снова из fd 4 и так далее, не теряя состояние чтения для каждого файлового дескриптора.

Добавьте суффикс `_bonus.[c\h]` к файлам бонусной части.
Это значит, что помимо файлов обязательной части вы сдаёте в корне репозитория следующие 3 файла:

- `get_next_line_bonus.c`
- `get_next_line_bonus.h`
- `get_next_line_utils_bonus.c`

> ⚠️ Бонусная часть будет оцениваться только в том случае, если обязательная часть выполнена идеально. «Идеально» означает, что обязательная часть выполнена полностью и работает без сбоев. Если вы не выполнили ВСЕ обязательные требования, бонусная часть не будет оцениваться вообще.

Глава VII — Сдача и взаимная проверка

Сдайте задание в свой `Git`-репозиторий как обычно. Во время защиты оценивается только содержимое вашего репозитория. Обязательно перепроверьте правильность названий файлов.

> ℹ️ При написании тестов помните, что:
> 1) И размер буфера, и длина строки могут принимать самые разные значения.
> 2) Файловый дескриптор может указывать не только на обычные файлы.
>
> Будьте внимательны и перепроверяйте свою работу вместе с коллегами. Подготовьте к защите обширный набор разнообразных тестов.

После успешной сдачи не стесняйтесь добавить свою `get_next_line()` в свою `libft`.

Во время проверки иногда может потребоваться небольшая **модификация проекта**. Это может быть незначительное изменение поведения, несколько строк кода, которые нужно написать или переписать, или легко добавляемая функциональность.

Хотя этот этап **может применяться не ко всем проектам**, вы должны быть к нему готовы, если он упомянут в критериях проверки.

Этот этап нужен, чтобы проверить ваше реальное понимание конкретной части проекта. Модификацию можно выполнить в любой среде разработки на ваш выбор (например, в вашей обычной), и она должна занимать несколько минут — если только в рамках проверки не установлено конкретное ограничение по времени.

Например, вас могут попросить внести небольшое изменение в функцию или скрипт, изменить вывод на экран или доработать структуру данных, чтобы она хранила новую информацию, и т. д.

Подробности (объём, цель и т. д.) будут указаны в **критериях проверки (evaluation guidelines)** и могут отличаться от одной проверки к другой для одного и того же проекта.